

深圳大学实验报告

课程名称： 编译原理

实验项目名称： 自底向上的语法分析程序设计

学院： 计算机与软件学院

专业： 软件工程

指导教师： 王祎乐

报告人： 学号： 班级：

实验时间： 2026 年 5 月 23 日至 6 月 7 日

实验报告提交时间： 2026 年 5 月 31 日

教务处制

实验目的与要求：

目的：通过设计、实现基于 LR 分析法的语法分析程序，掌握并能应用自底向上的语法分析技术进行语法分析。

要求：

第一部分：LR (0) 分析表项目的闭包

输入：一个 2 型文法

输出：拓广输入文法（新增 $S' \rightarrow S$ 产生式），并求项目集的闭包（ $I_0, I_1 \dots$ ）。

第二部分：LR(0)分析表的构造

输入：一个 2 型文法

输出：LR (0) 分析表（具体格式不要求，但是要能够体现 ACTION 和 GOTO，以及在对应情况下该执行的动作）

注：前两问 Input 和 Output 示例放在 grammar*.txt 和 example*.txt 里面了，其中 2 号是产生冲突的例子

第三部分：基于 LR(0)分析表的语法分析

输入：一个 2 型文法以及一个待分析的字符串

输出：利用 LR(0)分析表，给出对于给定字符串采用该文法进行规约的过程（体现每一步在每一步状态栈以及符号栈中有什么内容，以及在该情况下该进行什么操作）

第四部分：（选做）LR (1) 分析表的构造

输入：一个 2 型文法

输出：LR (1) 分析表；

方法、步骤：

要完成本实验，依据实验要求进行分解，需要完成的实验步骤是：

请在这里，参考前序实验方法、步骤部分的内容，补充完善你实验过程的具体步骤。

1. （选做）LR (0) 项目的存储设计

在底向上的 LR 语法分析程序中，单条项目需要反映两个关键属性：其所关联的产生式以及当前圆点（dot）在产生式右部的所处位置。为了保证后续规范项目集族推导中状态的唯一性判定与无多余开销的重复性过滤，项目的数据结构必须支持精准的高效去重和快速查找。

2. （选做）LR (0) 分析表的设计

LR 分析表是语法分析的核心控制矩阵，由行（对应活前缀 DFA 状态集编号）和列（终结符与非终结符）相交处的具体动作构成。由于实际编译器设计中状态空间和符号空间通常很大，但绝大部分格子都为空（即代表语法错误），因此若采用全尺寸二维数组存储将造成较大的内存冗余。

3. LR 分析过程中，移进动作的设计实现？

在底向上的 LR 分析器中，移进（Shift）动作由活前缀 DFA 的状态迁移所决定。其核心物理意义在于：当前符号栈和状态栈的格局允许接收当前的输入标记，将其作为当前正在匹配的某一产生式前缀的下一个符号推入，并将语法分析器状态转移到进一步推进后的匹配阶段。

4. LR 分析过程中，归约动作的设计实现？

归约 (Reduce) 是自底向上语法分析中建立语法树抽象节点的关键操作。其物理意义是：当符号栈顶的内容刚好完整拼凑出某一个产生式 $A \rightarrow \gamma$ 的右部短语 γ 时，分析器应将这一短语从栈顶弹出，并回退到未移入该短语之前的初始父状态，随后利用产生式左部的非终结符 A 通过 GOTO 关系定位并转移到父状态对应的后继转移状态。

实验过程及内容：

1. （选做）LR (0) 项目的存储设计

①单条产生式的表达 Production:

包含唯一的编号 id (用于区分相同右部的不同产生式)、非终结符左部 lhs、以及分词后的终结符与非终结符右部向量 rhs (vector<string>)。通过定义特定的产生式打印方式，可以方便地追踪归约动作。

②单个项目结构体 Item:

由产生式对象 prod 与整型圆点位置 dot_pos 组成。dot_pos 标识圆点左侧已有多少个符号，取值范围为 $[0, |Rhs|]$ 。圆点当前指向的符号通过函数 next_symbol() 获取，它返回 prod.rhs[dot_pos] (若 dot 已达末尾，则返回空串)。

③严格弱序判定与去重设计 (Comparators):

std::set 要求集合中元素重载严格弱序运算符 operator<。在 Item 中，首要排序依据是关联产生式的唯一编号 prod.id; 若编号一致，则由 dot_pos 的升序关系决定次序。相等性判定 operator== 也采用相同的多关键字组合判断机制。这保证了项目集 ItemSet 在自动排重和基于字典序的高效红黑树节点比较中保持唯一。

④空归约 (Epsilon) 项目的兼容:

对于右部为空或为 "epsilon" 的产生式，其初始状态和唯一状态均表现为圆点在起始位置 0 处，即 $[A \rightarrow \cdot]$ ，dot 的定位和取词逻辑被正确处理以避免发生指针溢出。

```
14
15 // Represents a production rule: Lhs -> Rhs
16 struct Production {
17     string lhs;
18     vector<string> rhs;
19     int id; // Unique ID for the production
20
21     bool operator<(const Production& other) const {
22         if (lhs != other.lhs) return lhs < other.lhs;
23         if (rhs != other.rhs) return rhs < other.rhs;
24         return id < other.id;
25     }
26
27     bool operator==(const Production& other) const {
28         return lhs == other.lhs && rhs == other.rhs && id == other.id;
29     }
30
31     解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
32     void print() const {
33         cout << id << ". " << lhs << " -> ";
34         if (rhs.empty() || (rhs.size() == 1 && rhs[0] == "epsilon")) { // epsilon often represented by empty or "epsilon"
35             cout << "epsilon";
36         } else {
37             for (const auto& s : rhs) {
38                 cout << s << " ";
39             }
40         }
41     };
42 }
```

```

43 // Represents an LR(0) item: Production with a dot
44 struct Item {
45     Production prod;
46     int dot_pos;
47
48     Item(Production p, int d) : prod(p), dot_pos(d) {}
49
50     bool operator<(const Item& other) const {
51         if (prod.id != other.prod.id) return prod.id < other.prod.id;
52         return dot_pos < other.dot_pos;
53     }
54
55     bool operator==(const Item& other) const {
56         return prod.id == other.prod.id && dot_pos == other.dot_pos;
57     }
58
59     void print() const {
60         cout << "[" << prod.lhs << " -> ";
61         for (int i = 0; i < prod.rhs.size(); ++i) {
62             if (i == dot_pos) cout << ".";
63             cout << prod.rhs[i] << " ";
64         }
65         if (dot_pos == prod.rhs.size()) cout << ".";
66         if (prod.rhs.empty() && dot_pos == 0) cout << "."; // For A -> .
67         cout << "]";
68     }
69
70     string next_symbol() const {
71         if (dot_pos < prod.rhs.size()) {
72             return prod.rhs[dot_pos];
73         }
74         return ""; // Empty string if dot is at the end
75     }
76 };
77
78 ...

```

2. （选做）LR（0）分析表的设计

①ACTION 动作表设计：

定义嵌套映射关系 `std::map<std::pair<int, string>, Action> action_table`。键是由当前的整型状态编号 `state_id` 和输入字符 `terminal` 组成的对。其值 `Action` 是一个包含动作类型枚举（SHIFT、REDUCE、ACCEPT、ERROR、CONFLICT）和动作参数 `value` 的自定义结构体。

在构建时，如果项目集包含移进型项目 $[A \rightarrow \alpha \cdot a\beta]$ ，我们直接通过 `GoTo` 关系的映射，将其转换到移进的状态。如果项目集包含可归约项目 $[A \rightarrow \alpha \cdot]$ ，则对每一个定义的终结符在 `action_table` 中插入归约记录。

②GOTO 转移表设计：

定义 `std::map<std::pair<int, string>, int> goto_table`，其键为 $\{state_id, non_terminal\}$ ，其值直接为下一个转移的目标状态 ID。在进行状态集规范族拓展时，自动抓取由非终结符带动的转换关系填充该表。

③冲突检测机制：

在向 `action_table` 填充元素前，程序会通过 `count` 进行既有键判定。如果目标表项已被填入其他不完全等同的动作（例如在处理 LR(0) 归约时，该行某列已填入 SHIFT 行动，或者存在其他产生式的 REDUCE），则说明当前格子被多次写，即发生了典型的移进-归约冲突或归约-归约冲突。此时程序会详细报错并标识具体发生冲突的状态、符号与冲突动作类型。

```

80 // Action in the parsing table
81 struct Action {
82     enum Type { SHIFT, REDUCE, ACCEPT, ERROR, CONFLICT } type;
83     int value; // State number for SHIFT, Production ID for REDUCE
84
85     解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
    Action() : type(ERROR), value(-1) {}
86     解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
    Action(Type t, int v) : type(t), value(v) {}
87
88     解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
    void print() const {
89         switch (type) {
90             case SHIFT: cout << "s" << value; break;
91             case REDUCE: cout << "r" << value; break;
92             case ACCEPT: cout << "acc"; break;
93             case ERROR: cout << "err"; break;
94             case CONFLICT: cout << "conf"; break; // Or print specific conflict
95         }
96     }
97 };
98

```

3. LR 分析过程中，移进动作的设计实现？

①触发机制：

当分析驱动器主循环执行时，它获取状态栈的栈顶元素 S_{curr} 以及输入缓冲区首部的 Token a 。通过联合主控键 $\{S_{curr}, a\}$ 检索 `action_table`。如果取得的行动对象其类型 `type` 为 `Action::SHIFT`，则移进动作被激活，动作指示移至目标状态 j （存储在 `Action::value` 中）。

②状态与符号栈变更：

将目标状态编号 j （表示接收到终结符 a 后的全新活前缀认知阶段）压入状态栈 `state_stack`。将终结符 Token a 本身压入符号栈 `symbol_stack`。

③输入指针递增（消耗符号）：

分析指针 `ip` 向右移动一位，表示成功消费该输入标记，接下来将查看新的 `lookahead`。

④跟踪记录与过程输出：

移进过程完毕后，程序输出本次“Shift j ”的简明调试日志。由于移进将一个全新的终结符推入栈，此时程序并不消耗栈内已有的任何内容，因而栈深度必然增加 1，这构成了 LR 语法树子节点逐步生成的前缀序列积累过程。

```

322 // Case 1: Shift action [A -> α . a β] where a is a terminal
323 string next_sym = item.next_symbol();
324 if (!next_sym.empty() && terminals.count(next_sym)) {
325     ItemSet next_state_candidate = goTo(state_items, next_sym);
326     if (state_to_id.count(next_state_candidate)) {
327         int j = state_to_id[next_state_candidate];
328         if (action_table.count({i, next_sym}) &&
329             (action_table[{i, next_sym}].type != Action::SHIFT || action_table[{i, next_sym}].value != j)) {
330             // Conflict!
331             cerr << "Conflict at ACTION[" << i << ", " << next_sym << "]: existing ";
332             action_table[{i, next_sym}].print();
333             cerr << ", new s" << j << endl;
334             action_table[{i, next_sym}].type = Action::CONFLICT;
335         } else {
336             action_table[{i, next_sym}] = Action(Action::SHIFT, j);
337         }
338     }
339 }

```

```

521
522     if (act.type == Action::SHIFT) {
523         cout << "Shift " << act.value << endl;
524         state_stack.push_back(act.value);
525         symbol_stack.push_back(a);
526         ip++;
527     } else if (act.type == Action::REDUCE) {
528         Production prod = productions[act.value];
529         cout << "Reduce using production " << prod.id << ": " << prod.lhs << " -> ";
530         if (prod.rhs.empty() || (prod.rhs.size() == 1 && prod.rhs[0] == "epsilon")) {
531             cout << "epsilon";
532         } else {
533             for (const auto& sym : prod.rhs) {
534                 cout << sym << " ";
535             }
536         }
537         cout << endl;

```

4. LR 分析过程中，归约动作的设计实现？

①触发与定位产生式：

分析器在特定状态 s 遇到输入符号 a ，查询 `action_table` 检索得到 REDUCE 动作，包含将要还原的产生式编号 k （由 `Action::value` 指定）。分析器通过编号在全局产生式数组 `productions` 中迅速获得产生式 $prod=(A \rightarrow X_1X_2...X_n)$ 。

②栈的回退（Pop 操作）：

产生式右部的符号个数为 $L=|Rhs|$ 。分析器必须同时从符号栈和状态栈顶弹出 L 个元素。这表明长度为 L 的子树前缀已匹配成功，并将在概念上缩并为其左部的语法节点 A 。注：特别地，若为 ϵ 归约（空产生式），由于其右部长度为 0，程序中设计逻辑为无需弹出任何符号和状态。

③非终结符的推入与状态跃迁（Goto 操作）：

在执行弹出后，状态栈的当前栈顶变更为之前尚未进入匹配该产生式时的历史节点状态 S_{prev} 。此时，分析器使用组合键 $\{S_{prev}, A\}$ 查询 `goto_table` 得到目标转移状态 S_{goto} 。接着，非终结符左部 A 被推入 `symbol_stack`，对应的状态 S_{goto} 被推入 `state_stack`。

④输入流指针状态：

归约不消费输入符号。因此，在归约操作完成后，输入指针 `ip` 保持原位，下一轮的 lookahead Token 依然是之前的符号 a 。这反映了在不改变未匹配输入流内容的前提下，符号栈内容正向高层抽象规整的过程。

```

527     } else if (act.type == Action::REDUCE) {
528         Production prod = productions[act.value];
529         cout << "Reduce using production " << prod.id << ": " << prod.lhs << " -> ";
530         if (prod.rhs.empty() || (prod.rhs.size() == 1 && prod.rhs[0] == "epsilon")) {
531             cout << "epsilon";
532         } else {
533             for (const auto& sym : prod.rhs) {
534                 cout << sym << " ";
535             }
536         }
537         cout << endl;
538
539         // Determine RHS pop length
540         int pop_len = prod.rhs.size();
541         if (prod.rhs.size() == 1 && prod.rhs[0] == "epsilon") {
542             pop_len = 0;
543         }
544
545         // Pop stacks
546         for (int k = 0; k < pop_len; ++k) {
547             if (!state_stack.empty()) state_stack.pop_back();
548             if (!symbol_stack.empty()) symbol_stack.pop_back();
549         }
550

```

```

551 // GOTO step
552 int prev_state = state_stack.back();
553 if (!goto_table.count({prev_state, prod.lhs})) {
554     cout << "Error: GOTO table entry not found for state " << prev_state << " and LHS " << prod.lhs << endl;
555     break;
556 }
557 int next_state = goto_table[{prev_state, prod.lhs}];
558 state_stack.push_back(next_state);
559 symbol_stack.push_back(prod.lhs);

```

运行结果:

1.测试用例 grammar1.txt

```
→ /workspace git:(master) X ./lr0_parser examples/grammar1.txt
```

```
--- Grammar Productions ---
```

```
0. S' -> S
```

```
1. S -> B B
```

```
2. B -> a B
```

```
3. B -> b
```

```
Non-terminals: B S S'
```

```
Terminals: $ a b
```

```
--- Building Canonical Collection of LR(0) Items ---
```

```
Processing State I0:
```

```
[S' -> . S ]
```

```
[S -> . B B ]
```

```
[B -> . a B ]
```

```
[B -> . b ]
```

```
Goto(I0, B) = I1 (New)
```

```
Goto(I0, S) = I2 (New)
```

```
Goto(I0, a) = I3 (New)
```

```
Goto(I0, b) = I4 (New)
```

```
Processing State I1:
```

```
[S -> B . B ]
```

```
[B -> . a B ]
```

```
[B -> . b ]
```

```
Goto(I1, B) = I5 (New)
```

```
Goto(I1, a) = I3
```

```
Goto(I1, b) = I4
```

```
Processing State I2:
```

```
[S' -> S .]
```

```
Processing State I3:
```

```
[B -> . a B ]
```

```
[B -> a . B ]
```

```
[B -> . b ]
```

```
Goto(I3, B) = I6 (New)
```

```
Goto(I3, a) = I3
```

```
Goto(I3, b) = I4
```

```
Processing State I4:
```

```
[B -> b .]
```

```
Processing State I5:
```

```
[S -> B B .]
```

```
Processing State I6:
```

```
[B -> a B .]
```

```
--- Building LR(0) Parsing Table ---
```

--- Building LR(0) Parsing Table ---

--- LR(0) Parsing Table ---

State	a	b	\$		B	S
0	s3	s4			1	2
1	s3	s4			5	
2			acc			
3	s3	s4			6	
4	r3	r3	r3			
5	r1	r1	r1			
6	r2	r2	r2			

States (Item Sets):

I0:

[S' -> . S]
[S -> . B B]
[B -> . a B]
[B -> . b]

I1:

[S -> B . B]
[B -> . a B]
[B -> . b]

I2:

[S' -> S .]

I3:

[B -> . a B]
[B -> a . B]
[B -> . b]

I4:

[B -> b .]

I5:

[S -> B B .]

I6:

[B -> a B .]

①文法拓广与符号识别

程序成功读取并对原始文法进行了处理:

文法拓广: 程序自动引入了拓广产生式 $\epsilon \cdot S' \rightarrow S'$, 以确保语法分析程序拥有单一的起始和接受状态。

非终结符识别: 成功识别出非终结符集合为 $\{B, S, S'\}$ 。

终结符识别: 成功识别出终结符集合为 $\{a, b, \$ \}$, 其中 $\$$ 为程序自动追加的输入流结束标记。

②活前缀 DFA 规范项目集族构造

初始状态 I_0 的推导:

初始核心项目为 $[S' \rightarrow \cdot S]$ 。由于圆点后为非终结符 S , 引入 $[S \rightarrow \cdot BB]$; 圆点后又遇到非终结符 B , 继续对其应用闭包运算, 引入 $[B \rightarrow \cdot aB]$ 和 $[B \rightarrow \cdot b]$ 。这四个项目共同构成了状态 I_0 的闭包集合。

状态间的转移演进:

I_0 接收非终结符 S 跳转至状态 I_2 : 包含项目 $[S' \rightarrow S \cdot]$, 此状态即为接受状态。

I_0 接收非终结符 B 跳转至状态 I_1 : 包含 $[S \rightarrow B \cdot B]$ 及其对右部 B 进行闭包展开引入的项目。 I_0 或 I_1 接收终结符 a 均跳转至状态 I_3 : 包含了圆点前移后的核心项目 $[B \rightarrow a \cdot B]$, 并再次对其圆点后的非终结符 B 展开闭包。

I_0 或 I_1 接收终结符 b 跳转至状态 I_4 : 仅包含一个纯归约项目 $[B \rightarrow b \cdot]$ 。

完全归约状态的形成:

规范族最终闭合于三个归约状态中: 状态 I_4 对应 $B \rightarrow b$ 归约 (项目为 $[B \rightarrow b \cdot]$); 状态 I_5 对应 $S \rightarrow BB$ 归约 (项目为 $[S \rightarrow BB \cdot]$); 状态 I_6 对应 $B \rightarrow aB$ 归约 (项目为 $[B \rightarrow aB \cdot]$)。

③LR(0) 分析表的构造

移进动作 (Shift):

当分析器处于状态 0、1、3 时, 若读入终结符 a 或 b, ACTION 表格对应位置填入 s3 或 s4 (如 ACTION[0, a] = s3), 表示将当前输入符号入符号栈, 并将状态 3 或 4 压入状态栈。

无条件归约动作 (Reduce):

符合纯 LR(0) 规则特性, 只要处于归约状态下, 系统不对前瞻符号 (lookahead) 做任何过滤, 即在当前行所有的终结符列上广播填入归约动作:

状态 4: 对 a, b, \$ 均填入 r3 (按第 3 条产生式 $B \rightarrow b$ 归约)。

状态 5: 对 a, b, \$ 均填入 r1 (按第 1 条产生式 $S \rightarrow BB$ 归约)。

状态 6: 对 a, b, \$ 均填入 r2 (按第 2 条产生式 $B \rightarrow aB$ 归约)。

状态跃迁 (Goto):

当发生归约并回退对应栈帧后, 分析器依据当前的栈顶状态和非终结符查询 GOTO 部分。

例如从状态 0 归约出非终结符 S 时, GOTO[0, S] = 2 指引分析器转移至状态 2。

接受判定 (Accept):

在状态 2 遇到结束符 \$ 时, ACTION[2, \$] = acc, 表示句子解析成功。

2.测试用例 grammar2.txt

```
Processing State I3:
[T -> F .]

Processing State I4:
[E -> T .]
[T -> T . * F ]
Goto(I4, *) = I8 (New)

Processing State I5:
[F -> id .]

Processing State I6:
[E -> E . + T ]
[F -> ( E . ) ]
Goto(I6, )) = I9 (New)
Goto(I6, +) = I7

Processing State I7:
[E -> E + . T ]
[T -> . T * F ]
[T -> . F ]
[F -> . ( E ) ]
[F -> . id ]
Goto(I7, () = I1
Goto(I7, F) = I3
Goto(I7, T) = I10 (New)
Goto(I7, id) = I5

Processing State I8:
[T -> T * . F ]
[F -> . ( E ) ]
[F -> . id ]
Goto(I8, () = I1
Goto(I8, F) = I11 (New)
Goto(I8, id) = I5

Processing State I9:
[F -> ( E ) .]

Processing State I10:
[E -> E + T .]
[T -> T . * F ]
Goto(I10, *) = I8

Processing State I11:
[T -> T * F .]

--- Grammar Productions ---
0. S' -> E
1. E -> E + T
2. E -> T
3. T -> T * F
4. T -> F
5. F -> ( E )
6. F -> id
Non-terminals: E F S' T
Terminals: $ ( ) * + id

--- Building Canonical Collection of LR(0) Items ---
Processing State I0:
[S' -> . E ]
[E -> . E + T ]
[E -> . T ]
[T -> . T * F ]
[T -> . F ]
[F -> . ( E ) ]
[F -> . id ]
Goto(I0, () = I1 (New)
Goto(I0, E) = I2 (New)
Goto(I0, F) = I3 (New)
Goto(I0, T) = I4 (New)
Goto(I0, id) = I5 (New)

Processing State I1:
[E -> . E + T ]
[E -> . T ]
[T -> . T * F ]
[T -> . F ]
[F -> . ( E ) ]
[F -> . ( E ) ]
[F -> . id ]
Goto(I1, () = I1
Goto(I1, E) = I6 (New)
Goto(I1, F) = I3
Goto(I1, T) = I4
Goto(I1, id) = I5

Processing State I2:
[S' -> E .]
[E -> E . + T ]
Goto(I2, +) = I7 (New)
```

--- Building LR(0) Parsing Table ---

Conflict at ACTION[4, *]: existing r2, new s8

Conflict at ACTION[10, *]: existing r1, new s8

--- LR(0) Parsing Table ---

State	(	)	*	+	id	\$		E	F	T
0	s1				s5			2	3	4
1	s1				s5			6	3	4
2				s7		acc				
3	r4	r4	r4	r4	r4	r4				
4	r2	r2	conf	r2	r2	r2				
5	r6	r6	r6	r6	r6	r6				
6		s9		s7						
7	s1				s5				3	10
8	s1				s5				11	
9	r5	r5	r5	r5	r5	r5				
10	r1	r1	conf	r1	r1	r1				
11	r3	r3	r3	r3	r3	r3				

States (Item Sets):

I0:

[S' -> . E]
[E -> . E + T]
[E -> . T]
[T -> . T * F]
[T -> . F]
[F -> . (E)]
[F -> . id]

I1:

[E -> . E + T]
[E -> . T]
[T -> . T * F]
[T -> . F]
[F -> . (E)]
[F -> (. E)]
[F -> . id]

I2:

[S' -> E .]
[E -> E . + T]

I3:

[T -> F .]

I4:

[E -> T .]
[T -> T . * F]

I5:

[F -> id .]

I6:

[E -> E . + T]
[F -> (E .)]

I7:

[E -> E + . T]
[T -> . T * F]
[T -> . F]
[F -> . (E)]
[F -> . id]

I8:

[T -> T * . F]
[F -> . (E)]
[F -> . id]

I9:

[F -> (E) .]

I10:

[E -> E + T .]
[T -> T . * F]

I11:

[T -> T * F .]

①文法拓广与符号识别

程序成功读取并对原始文法进行了处理：

文法拓广：程序自动引入了拓广产生式 $0. S' \rightarrow E$ ，以确保语法分析程序拥有单一的起始和接受状态。

非终结符识别：成功识别出非终结符集合为 $\{E, F, S', T\}$ 。

终结符识别：成功识别出终结符集合为 $\{(,), *, +, id, \backslash\}$ ，其中 $\backslash$ 为程序自动追加的输入流结束标记。

②活前缀 DFA 规范项目集族构造

程序成功计算并生成了 12 个规范状态 (I_0 至 I_{11})。

其中，有两个关键的状态项目集构成是引发分析表冲突的核心：

状态 I_4

包含项目组：

$[E \rightarrow T \cdot]$ (由产生式 2 归约的项目)

$[T \rightarrow T \cdot * F]$ (期待移进运算符 $*$ 的项目)

状态 I_{10}

包含项目组：

$[E \rightarrow E + T \cdot]$ (由产生式 1 归约的项目)

$[T \rightarrow T \cdot * F]$ (期待移进运算符 $*$ 的项目)

③语法分析表的冲突检测

冲突点一：ACTION[4, *]

冲突起因：状态 I_4 同时包含归约项 $[E \rightarrow T \cdot]$ 和移进项 $[T \rightarrow T \cdot * F]$ 。

冲突表现：LR (0) 分析法不看前瞻符号，会在该状态对所有终结符广播写入归约动作 r_2 ，但输入符号 $*$ 又可触发移进动作 s_8 。

结果：ACTION[4, *] 发生移进 s_8 与归约 r_2 冲突，标记为 $conf$ 。

冲突点二：ACTION[10, *]

冲突起因：状态 I_{10} 同时包含归约项 $[E \rightarrow E + T \cdot]$ 与移进项 $[T \rightarrow T \cdot * F]$ 。

冲突表现：输入符号 $*$ 既可以触发 r_1 归约，也可以触发 s_8 移进。

结果：ACTION[10, *] 发生移进 s_8 与归约 r_1 冲突，标记为 $conf$ 。

3.测试用例 grammar3.txt

```
● → /workspace git:(master) X ./lr0_parser examples/grammar3.txt
--- Grammar Productions ---
0. S' -> S
1. S -> c A
2. S -> c c B
3. A -> c A
4. A -> a
5. B -> c c B
6. B -> b
Non-terminals: A B S S'
Terminals: $ a b c
```

--- Building Canonical Collection of LR(0) Items ---

Processing State I0:
 [S' -> . S]
 [S -> . c A]
 [S -> . c c B]
 Goto(I0, S) = I1 (New)
 Goto(I0, c) = I2 (New)

Processing State I1:
 [S' -> S .]

Processing State I2:
 [S -> c . A]
 [S -> c . c B]
 [A -> . c A]
 [A -> . a]
 Goto(I2, A) = I3 (New)
 Goto(I2, a) = I4 (New)
 Goto(I2, c) = I5 (New)

Processing State I3:
 [S -> c A .]

Processing State I4:
 [A -> a .]

Processing State I5:

[S -> c c . B]
 [A -> . c A]
 [A -> c . A]
 [A -> . a]
 [B -> . c c B]
 [B -> . b]
 Goto(I5, A) = I6 (New)
 Goto(I5, B) = I7 (New)
 Goto(I5, a) = I4
 Goto(I5, b) = I8 (New)
 Goto(I5, c) = I9 (New)

Processing State I6:

[A -> c A .]

Processing State I7:

[S -> c c B .]

Processing State I8:

[B -> b .]

Processing State I9:

[A -> . c A]
 [A -> c . A]
 [A -> . a]
 [B -> c . c B]
 Goto(I9, A) = I6
 Goto(I9, a) = I4
 Goto(I9, c) = I10 (New)

Processing State I10:

[A -> . c A]
 [A -> c . A]
 [A -> . a]
 [B -> . c c B]
 [B -> c c . B]
 [B -> . b]
 Goto(I10, A) = I6
 Goto(I10, B) = I11 (New)
 Goto(I10, a) = I4
 Goto(I10, b) = I8
 Goto(I10, c) = I9

Processing State I11:

[B -> c c B .]

--- Building LR(0) Parsing Table ---

--- LR(0) Parsing Table ---

State	a	b	c	\$		A	B	S
0			s2					1
1				acc				
2	s4		s5			3		
3	r1	r1	r1	r1				
4	r4	r4	r4	r4				
5	s4	s8	s9			6	7	
6	r3	r3	r3	r3				
7	r2	r2	r2	r2				
8	r6	r6	r6	r6				
9	s4		s10			6		
10	s4	s8	s9			6	11	
11	r5	r5	r5	r5				

States (Item Sets):

I0:

[S' -> . S]
[S -> . c A]
[S -> . c c B]

I1:

[S' -> S .]

I2:

[S -> c . A]
[S -> c . c B]
[A -> . c A]
[A -> . a]

I3:

[S -> c A .]

I4:

[A -> a .]

I5:

[S -> c c . B]
[A -> . c A]
[A -> c . A]
[A -> . a]
[B -> . c c B]
[B -> . b]

I6:

[A -> c A .]

I7:

[S -> c c B .]

I8:

[B -> b .]

I9:

[A -> . c A]
[A -> c . A]
[A -> . a]
[B -> c . c B]

I10:

[A -> . c A]
[A -> c . A]
[A -> . a]
[B -> . c c B]
[B -> c c . B]
[B -> . b]

I11:

[B -> c c B .]

①文法属性与符号识别

程序成功读取并对原始文法进行了处理:

拓广产生式: 生成了 0. $S' \rightarrow S$ 。

非拓广产生式:

1: $S \rightarrow cA$

2: $S \rightarrow ccB$

3: $A \rightarrow cA$

4: $A \rightarrow a$

5: $B \rightarrow ccB$

6: $B \rightarrow b$

非终结符集合: {A, B, S, S'}。

终结符集合: {a, b, c, \$} (\$ 为自动追加的结束标记)。

②项目集规范族

状态 I_2 的分流:

在初始状态 I_0 遇到第一个字符 c 时, 转移至状态 I_2 :

项目包含: $[S \rightarrow c \cdot A]$ 与 $[S \rightarrow c \cdot cB]$, 并自动扩充非终结符 A 的闭包项 $[A \rightarrow \cdot cA]$ 和 $[A \rightarrow \cdot a]$ 。

在 I_2 遇到第二个字符 c 时, 会转移至状态 I_5 。

高复杂度状态 I_5 的构成:

由于连续扫描到两个 c, 分析器走到了分支合并区:

核心项目: $[S \rightarrow cc \cdot B]$ (期待非终结符) 与 $[A \rightarrow c \cdot A]$ (期待非终结符 A)。

由于圆点后有非终结符, 程序自动拉入闭包: $[A \rightarrow \cdot cA]$ 、 $[A \rightarrow \cdot a]$ 、 $[B \rightarrow \cdot ccB]$ 、 $[B \rightarrow \cdot b]$ 。

在状态 I_5 处, 程序精确完成了“五向分流”:

遇到 A 转移至状态 I_6 ;

遇到 B 转移至状态 I₇;
遇到 a 转移至状态 I₄;
遇到 b 转移至状态 I₈;
遇到 c 转移至状态 I₉。

③语法分析表的构造特征

移进 / 状态跳转:

状态 0、2、5、9、10 均含有向后续状态移进的动作。例如: 在状态 5 遇到输入符号 c 时, 查表执行 s₉ (移进并跳转到状态 9)。

全终结符广播归约:

由于该文法是无冲突的 LR (0) 类, 因此所有完备归约状态均在所有终结符上触发相同的归约动作:

状态 3: 完备项目 $[S \rightarrow cA \cdot] \rightarrow$ 全列填入 r₁。

状态 4: 完备项目 $[A \rightarrow a \cdot] \rightarrow$ 全列填入 r₄。

状态 6: 完备项目 $[A \rightarrow cA \cdot] \rightarrow$ 全列填入 r₃。

状态 7: 完备项目 $[S \rightarrow ccB \cdot] \rightarrow$ 全列填入 r₂。

状态 8: 完备项目 $[B \rightarrow b \cdot] \rightarrow$ 全列填入 r₆。

状态 11: 完备项目 $[B \rightarrow ccB \cdot] \rightarrow$ 全列填入 r₅。

实验结论:

1. (选做) 为了验证你 LR(0)分析表的构造结果是准确的, 你设计了什么测试数据 (2-3 个) 进行测试, 得到的结果如何。

(1) ex1

```
ser.cpp U  ex1.txt U X
testEx > ex1.txt
1 S -> ( L )
2 S -> x
3 L -> L , S
4 L -> S
```

运行结果:

```
➔ /workspace git:(master) X ./lr0_parser testEx/ex1.txt
--- Grammar Productions ---
0. S' -> S
1. S -> ( L )
2. S -> x
3. L -> L , S
4. L -> S
Non-terminals: L S S'
Terminals: $ ( ) , x
```



```

States (Item Sets):
I0:
  [S' -> . S ]
  [S -> . ( L ) ]
  [S -> . x ]
I1:
  [S -> ( . L ) ]
I2:
  [S' -> S .]
I3:
  [S -> x .]
I4:
  [S -> . ( L ) ]
  [S -> ( . L ) ]
  [S -> . x ]
  [L -> . L , S ]
  [L -> . S ]
I5:
  [S -> ( L . ) ]
  [L -> L . , S ]
I6:
  [L -> S .]
I7:
  [S -> ( L . ) ]
  [L -> L . , S ]
I8:
  [S -> ( L ) .]
I9:
  [L -> L , . S ]
I10:
  [S -> . ( L ) ]
  [S -> . x ]
  [L -> L , . S ]
I11:
  [L -> L , S .]

```

①文法特殊解析

根据代码中的 `tokenize_rhs` 函数，程序采用的是除 "id" 之外的单字符切分模式。由于输入的 `ex1.txt` 文件中，产生式右部符号之间留有空格（例如 `S -> ( L )`），导致程序将空格字符 " " 识别为了一个合法的终结符。

终结符集合：输出的 `Terminals:` 列表中，在 `$` 和 `(` 之间显式存在一个空格。

ACTION 分析表表头：在 `State` 列与 `(` 列之间，多出了一个空白字符列，该列即代表空格终结符 " "。

跳转显示：如 `I1` 状态下的转移 `Goto(I1, ) = I4`，括号中间其实是一个空格符号。

②项目集规范族（DFA）推导轨迹

文法被拓广为 5 条产生式（含拓广项 $S' \rightarrow S$ ）。程序成功推导出 12 个规范状态（`I0` 至 `I11`），核心状态转移轨迹如下：

初始状态 `I0`：

包含 $[S' \rightarrow \cdot S]$ 及其闭包项目。此时若读入左括号 `(`，程序执行移进动作，转移至状态 `I1`。

状态 `I1` 与状态 `I4`：

在状态 `I1` 中，项目为 $[S -> (\cdot L)]$ （注意圆点后是一个空格终结符）。

当分析器遇到空格字符 " " 时，执行移进动作，转移到新状态 `I4`。

在状态 `I4` 中，圆点越过了空格： $[S -> (\cdot L)]$ 。由于圆点后紧跟非终结符 `L`，程序自动计算闭包，引入了 $[L \rightarrow \cdot L, S]$ 和 $[L \rightarrow \cdot S]$ 及其级联闭包。这一步闭包扩展完全正确。

状态 `I5`、`I7` 与 `I8`：

自 `I4` 读入非终结符 `L` 转移至状态 `I5` ($[S -> (L \cdot)]$)。

在 I_5 遇到空格字符 " ", 转移至 I_7 ($[S \rightarrow (L \cdot)]$)。

在 I_7 遇到右括号), 转移至 I_8 ($[S \rightarrow (L) \cdot]$)。这是一个完备归约状态, 对应产生式 1。

状态 I_9 、 I_{10} 与 I_{11} :

当在 I_7 遇到逗号 , 时, 转移至状态 I_9 , 进而遇到空格转移至状态 I_{10} 。在 I_{10} 遇到非终结符 S 后转移至状态 I_{11} , 形成另一个完备归约状态 $[L \rightarrow L, S \cdot]$ (对应产生式 3)。

③LR(0) 分析表行为及无冲突验证

移进-归约决策清晰:

状态 1: 在空格列 (第一列) 填入了 s4, 表示在 (后遇到空格, 移进并跳转到状态 4。

状态 2: 在结束符 \$ 列填入了 acc, 宣告分析成功。

状态 3: 完备项目 $[S \rightarrow x \cdot]$, 无视前瞻符号, 在所有终结符列上统一填入 r2。

状态 6: 完备项目 $[L \rightarrow S \cdot]$, 所有终结符列统一填入 r4。

状态 8: 完备项目 $[S \rightarrow (L) \cdot]$, 所有终结符列统一填入 r1。

状态 11: 完备项目 $[L \rightarrow L, S \cdot]$, 所有终结符列统一填入 r3。

无冲突特性:

整张大表没有任何多重写覆盖, 不存在任何移进-归约或归约-归约冲突。

(2) ex2

```
ser.cpp U  ex1.txt U  ex2.txt U X
testEx > ex2.txt
1  S -> a S b
2  S -> a
```

运行结果:

```
➔ /workspace git:(master) X ./lr0_parser testEx/ex2.txt
--- Grammar Productions ---
0. S' -> S
1. S -> a S b
2. S -> a
Non-terminals: S S'
Terminals: $ a b
```

--- Building Canonical Collection of LR(0) Items ---

Processing State I0:

```
[S' -> . S ]
[S -> . a S b ]
[S -> . a ]
Goto(I0, S) = I1 (New)
Goto(I0, a) = I2 (New)
```

Processing State I1:

```
[S' -> S .]
```

Processing State I2:

```
[S -> a . S b ]
[S -> a .]
Goto(I2, ) = I3 (New)
```

Processing State I3:

```
[S -> . a S b ]
[S -> a . S b ]
[S -> . a ]
Goto(I3, S) = I4 (New)
Goto(I3, a) = I2
```

Processing State I4:

```
[S -> a S . b ]
Goto(I4, ) = I5 (New)
```

Processing State I5:

```
[S -> a S . b ]
Goto(I5, b) = I6 (New)
```

Processing State I6:

```
[S -> a S b .]
```

--- Building LR(0) Parsing Table ---

Conflict at ACTION[2,]: existing s3, new r2

--- LR(0) Parsing Table ---

State		a	b	\$		S
0		s2				1
1				acc		
2	conf	r2	r2	r2		
3		s2				4
4	s5					
5			s6			
6	r1	r1	r1	r1		

States (Item Sets):

I0:

```
[S' -> . S ]
[S -> . a S b ]
[S -> . a ]
```

I1:

```
[S' -> S .]
```

I2:

```
[S -> a . S b ]
[S -> a .]
```

I3:

```
[S -> . a S b ]
[S -> a . S b ]
[S -> . a ]
```

I4:

```
[S -> a S . b ]
```

I5:

```
[S -> a S . b ]
```

I6:

```
[S -> a S b .]
```

①项目集规范族 (DFA) 推导与状态 I_2 分析

程序成功推导出 7 个规范状态 (I_0 至 I_6)，其中状态 I_2 是整个分析器的核心决策分水岭： I_2 的项目集：

项目 1: $[S \rightarrow a \cdot S b]$

根据单字符切分逻辑，圆点 $\cdot$ 后面紧跟的是一个空格终结符 $" "$ 。因此，若面临输入符号为 $" "$ ，分析器应当执行**移进 (Shift)**动作，跳转到状态 I_3 (即分析表中写入 $s3$)。

项目 2: $[S \rightarrow a \cdot]$

这是一个已经匹配完毕的完备归约项目。按照纯 LR(0) 归约规则，分析器应当无条件在所有终结符 (包括 $a, b, \$$ 以及空格 $" "$) 上执行**归约 (Reduce)**动作，采用第 2 条产生式 $S \rightarrow a$ 进行归约 (即写入 $r2$)。

②移进-归约冲突的捕获与输出

由于状态 I_2 同时存在上述两个项目，在填充分析表第 2 行的空格终结符 $" "$ 列时，程序捕获到了严重的移进-归约冲突：在状态 2 遇到空格时，分析器既可以移进 ($s3$)，又可以归约 ($r2$)，决策产生了二义性。

文法类型判定：由于在 $ACTION[2, " "]$ 处产生了 $conf$ 标志，该文法被准确地判定为非 LR(0) 文法。

冲突检测模块验证：与设计样例 1 成功构建无冲突表形成完美对照，设计样例 2 证实了本程序在面对非 LR(0) 文法时，能够敏锐地捕获到“移进-归约”冲突，并输出精确的冲突位置与动作详情 ($existing\ s3, new\ r2$)，分析表构造结果完全符合编译原理的理论预期，验证了分析表构造算法的高可靠度与准确性。

2. 为了验证 LR(0)分析过程是准确的，你设计了什么测试数据进行测试 (2-3 个)，得到的结果如何。

(1) 测试输入(x, x)

运行结果：

```
➡ /workspace git:(master) X ./lr0_parser testEx/ex1.txt "( x , x )"
--- Grammar Productions ---
0. S' -> S
1. S -> ( L )
2. S -> x
3. L -> L , S
4. L -> S
Non-terminals: L S S'
Terminals: $ ( ) , x
```


States (Item Sets):

I0:

[S' -> . S]
[S -> . (L)]
[S -> . x]

I1:

[S -> (. L)]

I2:

[S' -> S .]

I3:

[S -> x .]

I4:

[S -> . (L)]
[S -> (. L)]
[S -> . x]
[L -> . L , S]
[L -> . S]

I5:

[S -> (L .)]
[L -> L . , S]

I6:

[L -> S .]

I7:

[S -> (L .)]
[L -> L . , S]

I8:

[S -> (L) .]

I9:

[L -> L , . S]

I10:

[S -> . (L)]
[S -> . x]
[L -> L , . S]

I11:

[L -> L , S .]

Input String: (x , x)

阶段 1: 嵌套初始化与首元素识别

状态推进:

分析自初始状态 0 开始。

读入 (, 移进并跳转到状态 1。

读入空格 "", 移进并跳转到状态 4。此时, 符号栈内为 \$(, 进入了括号嵌套层。

读入元素 x, 移进并跳转到状态 3。

句柄归约:

状态 3 的项目集为完备项目 [S -> x .]。分析器识别出当前的句柄为 x, 果断执行归约:

状态栈弹出 3, 回退至状态 4。

符号栈弹出 x, 压入非终结符 S。

根据 GOTO[4, S] = 6, 状态 6 入栈 (步骤 4)。

链首建立:

在状态 6 ([L -> S .]) 处, 分析器立即应用单产生式归约动作 $L \rightarrow S$, 将首个元素转化为列表头。状态栈回退至 4, 查 GOTO[4, L] = 5。此时符号栈规整为 \$(L (步骤 5)。这完成了列表递归定义的基准情况 (Base Case)。

阶段 2: 列表元素追加与递归归约 (步骤 6 - 11)

随着列表首元素 L 确立, 分析器开始通过逗号算符引入后续元素:

算符移进:

分析器依次移进空格、逗号、空格: $I_5 \rightarrow I_7 \rightarrow I_9 \rightarrow I_{10}$ 。

至步骤 8 结束时, 符号栈累积为 \$(L,, 分析器成功进入了等待第二个元素的就绪状态 10。

次元素识别:

步骤 9 移进第二个 x 转移到状态 3; 步骤 10 再次执行 $S \rightarrow x$ 归约, 查 GOTO[10, S] = 11, 状态 11 入栈。此时, 符号栈呈现为 \$(L, S (步骤 10)。

列表递归结合:

状态 11 对应完备项目 [L -> L, S .]。分析器识别出列表的追加句柄 L, S 匹配成功, 执行大归约 $L \rightarrow L, S$:

双栈同时弹出 5 个元素 (L、空格、,、空格、S 及对应状态), 状态栈退回到深层的状态 4。

压入非终结符 L，查 GOTO[4, L] = 5，状态 5 重新入栈。

至此，列表内容成功缩并为统一的列表非终结符 L

阶段 3：嵌套闭合与最终接收（步骤 12 - 15）

括号闭合：

处于状态 5 的分析器依次移进空格、右括号)，路径为 $I_5 \rightarrow I_7 \rightarrow I_8$ 。

至步骤 13 结束时，符号栈内呈现为完备的括号表达式 $\$(L)$ 。

整体归约：

状态 8 的项目集为完备项 $[S \rightarrow (L) \cdot]$ 。分析器执行大归约 $S \rightarrow (L)$ ：

双栈弹出 5 个元素（左括号、列表、右括号等及状态），状态栈回退到最外层状态 0。

压入非终结符 S，查询 GOTO[0, S] = 2，状态 2 入状态栈（步骤 14）。

成功接收：

在步骤 15 中，分析器在状态 2 读入结束标记 \$，查表执行 acc (Accept) 动作。分析器停止运行，宣告输入串 (x, x) 语法分析成功。

(2) 测试输入 (x,)

运行结果：

```
● → /workspace git:(master) X ./lr0_parser testEx/ex1.txt "( x , )"
--- Grammar Productions ---
0. S' -> S
1. S -> ( L )
2. S -> x
3. L -> L , S
4. L -> S
Non-terminals: L S S'
Terminals: $ ( ) , x
```

```
Processing State I5:
[S -> ( L . ) ]
[L -> L . , S ]
Goto(I5, ) = I7 (New)
```

Processing State I0:

```
Processing State I6:  
[L -> S .]
```

```
[S' -> . S ]
[S -> . (   L   ) ]
[S -> . x ]
Goto(I0, () = I1 (New)
Goto(I0, S) = I2 (New)
Goto(I0, x) = I3 (New)
```

```
Processing State I7:
[S -> ( L . ) ]
[L -> L . , S ]
Goto(I7, ) = I8 (New)
Goto(I7, , ) = I9 (New)
```

Processing State I1:

Processing State I8:
[S -> (L) .]

[S -> (. L)]
Goto(I1,) = I4 (New)

Processing State I2:

```
Processing State I9:  
[ L -> L    , .    S ]  
Goto(I9,    ) = I10 (New)
```

[S' -> S .]

Processing State I3:

```
Processing State I10:
  [S -> . ( L ) ]
  [S -> . x ]
  [L -> L , . S ]
Goto(I10, () = I1
Goto(I10, S) = I11 (New)
Goto(I10, x) = I3
```

[S → x.]

Processing State I4:

```
Processing State I11:
  [L -> L    ,   S .]
```

```
[S -> . ( L ) ]
[S -> ( . L ) ]
[S -> . x ]
[L -> . L , S ]
[L -> . S ]

Goto(I4, ( ) = I1
Goto(I4, L) = I5 (New)
Goto(I4, S) = I6 (New)
Goto(I4, x) = I3
```

```
--- Building LR(0) Parsing Table ---
```

--- LR(0) Parsing Table ---

State	(	)	,	x	\$	L	S
0		s1		s3			2
1	s4						
2					acc		
3	r2	r2	r2	r2	r2		
4		s1		s3		5	6
5	s7						
6	r4	r4	r4	r4	r4		
7		s8	s9				
8	r1	r1	r1	r1	r1		
9	s10						
10		s1		s3			11
11	r3	r3	r3	r3	r3		

```

States (Item Sets):
I0:
[S' -> . S ]
[S -> . ( L ) ]
[S -> . x ]
I1:
[S -> ( . L ) ]
I2:
[S' -> S .]
I3:
[S -> x .]
I4:
[S -> . ( L ) ]
[S -> ( . L ) ]
[S -> . x ]
[L -> . L , S ]
[L -> . S ]
I5:
[S -> ( L . ) ]
[L -> L . , S ]
I6:
[L -> S .]
I7:
[S -> ( L . ) ]
[L -> L . , S ]
I8:
[S -> ( L ) .]
I9:
[L -> L , . S ]
I10:
[S -> . ( L ) ]
[S -> . x ]
[L -> L , . S ]
I11:
[L -> L , S .]
Input String: ( x , )

```

①畸变句型的语法特征

在上下文无关文法中，输入串 (x,) 代表了一个尾随逗号错误 (Trailing Comma Error)。

根据文法规则 $L \rightarrow L, S$ ，在逗号 , 之后，语法强制期待另一个非终结符元素 S

(其可以推导出 (L) 或标识符 x)。在逗号后直接闭合右括号) 违反了产生式结构，属于典型的语法错误。

程序设计该用例的目的，是验证分析器在面对语法畸变时，能否准确阻断、不发生越界或死循环，并精准报告错误位置。

②分步推导追踪

分析器的输入 Token 序列为: {"(", " ", "x", " ", ",", " ", ")", "\$"}。

状态栈与符号栈的分步变化轨迹如下:

步骤 1 至 5 (前置子元素匹配):

分析过程与合法句型一致。左括号 (和首个元素 x 依次被移进，并被成功规约为非终结符 L，状态栈稳健推进。

此时，栈状态为:

状态栈: 0 1 4 5

符号栈: \$(L

步骤 6: 当前状态为 5，读入空格 " "。查表 ACTION[5, " "] = s7。空格入栈，状态 7 入栈。

步骤 7: 当前状态为 7，读入逗号 ,。查表 ACTION[7, ","] = s9。逗号入栈，状态 9 入栈。

步骤 8: 当前状态为 9，读入空格 " "。查表 ACTION[9, " "] = s10。空格入栈，状态 10 入栈。

此时双栈格局:

状态栈: 0 1 4 5 7 9 10

符号栈: \$(L, (逗号及空格已被成功消耗，分析器就绪，期待下一个元素)

分析器不仅成功拦截了非法输入，还极为精确地指出了出错发生时的状态 (State 10，即代表“列表逗号后期待元素”的状态) 以及导致语法崩溃的非法 Token (右括号))。

心得体会：

通过本次自底向上 LR (0) 语法分析程序设计实验，我完整实现了从文法拓广、项目集闭包计算、规范项目集族构造、LR (0) 分析表生成到基于分析表的移进归约语法分析全过程，不仅加深了对编译原理中自底向上语法分析理论的理解，也在代码实现与调试中提升了工程实践能力。

在理论认知上，我真正掌握了 LR (0) 分析法的核心思想：以活前缀为线索，通过构造确定有限自动机 DFA 识别所有规范句型的活前缀，再依据项目集状态与输入符号决定移进、归约、接受或报错动作。实验让我清晰理解了闭包运算与 GoTo 转移的意义 —— 闭包用于完整扩展项目集，GoTo 则实现状态间的合法跳转，二者共同构建出规范项目集族，为分析表构造提供基础。同时，我也直观认识到 LR (0) 文法的局限性：它不考虑前瞻符号，归约项目会对所有终结符执行归约，极易产生移进 - 归约冲突与归约 - 归约冲突，这也让我明白 SLR、LR (1) 等更高级分析法引入前瞻符的必要性。

在程序实现过程中，我深刻体会到数据结构设计对 LR 分析器的重要性。从 LR (0) 项目的结构体设计，到产生式、项目集、分析表的存储，每一个结构都直接影响算法效率与正确性。例如通过重载比较运算符实现项目去重、用映射表存储分析表以节省内存，这些细节让我学会用工程思维优化理论算法。而移进与归约动作的实现，让我彻底理解了状态栈与符号栈协同工作的逻辑：移进是将输入符号与新状态入栈，归约则是弹出句柄、回退状态后通过 GoTo 跳转，整个过程严格遵循自底向上的规约规则。

本次实验的调试过程也让我收获颇丰。在测试不同文法时，无冲突文法能顺利完成分析，而存在二义性的文法会精准触发冲突检测，这不仅验证了程序的正确性，也让我学会通过冲突位置定位文法问题。同时，对合法与非法输入串的分析测试，让我理解了 LR 分析器错误检测的原理，体会到语法分析在编译器中承担的 “语法校验” 核心作用。

指导教师批阅意见：

成绩评定：

指导教师签字：王祎乐
年 月 日

备注：